



可验证随机函数的多主节点共识算法

李怡霖, 黄慧*, 兰玉

(闽南师范大学计算机学院, 福建漳州 363000)

摘要: 共识算法是区块链技术的核心, PBFT(practical byzantine fault tolerance)是广泛应用于区块链的一类共识算法. 目前, 现有的PBFT存在着主节点选取公平性问题、吞吐量较小以及通信时延较高等问题. 为此, 提出了一种基于可验证随机函数 VRF(verifiable random function)的多主节点共识算法 MVBFT(multi-primary-node verifiable-random-function byzantine fault tolerance). MVBFT引入了共识阶的概念, 改进了区块链的区块结构, 实现流水线式共识. 其次, 采用信誉模型, 动态评估全网节点的信誉值. 利用节点的信誉值和可验证随机函数, 每个共识阶开始前随机选举出多个主节点, 提升系统的安全性和可靠性. 最后通过对PBFT算法的一致性协议进行改进, 调整了网络拓扑结构, 以减少通信量. 仿真实验结果表明, 与PBFT算法相比, 所提出的MVBFT算法具有更低的通信开销和更高吞吐量, 可以降低数据时延.

关键词: 共识算法; 信誉机制; 区块链; PBFT; VRF

中图分类号: TP391 **文献标志码:** A **文章编号:** 2095-7122(2024)02-0074-13

Multi-primary-node consensus algorithm based on verifiable random functions

LI Yilin, HUANG Hui*, LAN Yu

(School of Computer Science, Minnan Normal University, Zhangzhou, Fujian 363000, China)

Abstract: The consensus algorithm is the backbone of blockchain technology, PBFT (practical byzantine fault tolerance) is a kind of consensus algorithm widely used in blockchain. At present, the existing PBFT has some problems, such as the fairness of primary node selection, low throughput and high communication delay. A multi-primary-node consensus algorithm MVBFT (multi-primary-node verified-random-function byzantine fault tolerance) based on verifiable random function VRF (verifiable random function) is proposed. First, MVBFT introduces the concept of consensus order, improves the block structure of blockchain, and realizes pipelined consensus. Second, a reputation model is used to dynamically evaluate the reputation value of the whole network nodes. Multiple primary nodes are randomly selected before the beginning of each consensus order by using the reputation values of the nodes and a verifiable random function, so that the security and reliability of the system are improved. Simulations conducted to evaluate the performance of MVBFT demonstrate its superiority over PBFT. The results indicate lower communication overhead, higher throughput, and reduced data delay.

收稿日期: 2024-01-22

基金项目: 福建省自然科学基金项目(2023J01920)

作者简介: 李怡霖(1998—), 男, 河南汝州人, 硕士生.

*通信作者. E-mail: hhui323@163.com

Key words: consensus algorithm; reputation mechanism; blockchain; PBFT; VRF

区块链的去中心化、不可篡改、安全可靠等特性^[1],使其在多个领域得到了广泛应用^[2],为各行各业的数字化转型提供了更多的可能性.随着越来越多的政府、企业和个人将数据放在区块链网络中,对区块链技术的要求也日益提高,包括交易效率、安全性和信任度.共识算法是区块链的核心,能够解决在不可信网络中所有诚实节点达成共识的问题.研究共识算法对于推动区块链的发展至关重要.

共识算法根据对拜占庭节点容忍度的不同,共识算法被分为两类:BFT(byzantine fault tolerance)共识和非BFT共识^[3].Castro等^[4]在1999年提出的PBFT(practical byzantine fault tolerance)是经典的BFT共识,能够解决半同步环境中拜占庭故障问题^[5],是目前在区块链领域中广泛应用的共识算法之一.然而,该算法仍存在问题,如主节点选取公平性问题、吞吐量较小以及通信时延较高.为了提高PBFT算法的性能,学者对其进行了不断改进.

在提高PBFT算法的共识效率方面,Kotla等^[6]提出了SBFT算法,共识节点收到客户端的事务请求后,直接处理请求,不必先发送到共识网络中等待其他节点的验证,提高了系统效率.Li等^[7]提出了SHBFT算法,通过设置主节点选择和弹劾机制优化来提高共识效率,并使用新的网络拓扑结构.陈子豪等^[8]提出K-PBFT算法,根据特征对参与共识的大规模网络节点进行聚类与层次划分,减少通信量.高娜等^[9]提出NAC-PBFT算法,利用行业系统中网络结构引入种子节点,并进行自聚类分组,各组内部选出代理节点进行全局共识,提高共识效率.张雷^[10]提出了IMPBFT算法,融合了“无固定流水线周期”与“多阶段消息统一签名”两大机制,前者使得多轮共识得以并行执行,后者则有效减少了签名及验证签名的频次.这两种机制的完美结合,显著提升了共识效率.刘鑫磊^[11]提出了PCS-BFT算法,设计了链式与图式两种流水线结构.链式流水线通过精巧的并行处理机制,有序应对不同的交易出块请求,从而根治了并发出块可能导致的乱序问题,确保所有经验证的区块能依序形成链式结构;而图式流水线则采用了一种无领导者的全新设计,借助有向无环图优化出块排序,使所有验证通过的区块能以更为高效的图式结构呈现.这两种设计不仅各具特色,更在系统的吞吐量和共识效率上取得了显著提升.

在增强PBFT算法的安全性方面,Micali等^[12-15]提出了Algorand共识算法,该算法将PoS和可验证随机函数VRF相结合用于随机选取区块的领导者和验证者,使用BA*算法完成共识,能够抵抗DDoS攻击.任永旺等^[16]提出VRF_TS_BFT算法,通过将VRF和零知识证明相结合,实现了隐匿主节点身份的算法,再利用门限签名机制进行投票选举主节点,从而大大提高了安全性.王博超^[17]提出DV-Pbft算法,使用DPoS选出主节点和组长节点,使用VRF进行分组,使得分组结果具有不可预测性.黄保华等^[18]提出SBFT算法,通过采用改进的VRF来选举主节点,优化了一致性协议,从而有效地降低了通信开销,并提高了系统的安全性.

此外,还有在其他方面的改进,Xu等^[19]提出动态PBFT算法,添加了动态进出的协议,允许备份和节点加入或退出共识网络,其具有更好的动态性.张仕将等^[20]提出GBC算法,该算法采

用统一的数据结构,节点间通过 Gossip 协议进行通信,各节点通过计算其 GD 向量中的元素达成共识,提高了系统的拓展性。

这些改进的方案在提高主节点选举安全性、优化共识过程中的通信开销和交易时延,以及增强算法的拓展性方面取得了显著进展。然而,仍需关注一些尚未解决的问题,如主节点作恶对共识效率产生的影响以及通信开销仍然较高。此外,新出现的问题也需引起重视,如额外的通信开销、分组共识无法保证组内满足三分之一的容错率,以及系统稳定后可能出现的寡头现象降低公平性。因此,结合实际应用的需求研究更好的共识算法至关重要。

1 相关知识

1.1 区块链结构

研究学者从狭义和广义两个角度分别定义了区块链^[21]。从狭义的角度来说,区块链是一种不可篡改的链状分布式数据库;从广义的角度来说,区块链是一种融合了密码学、P2P 网络、共识算法和智能合约等技术的去中心化系统^[22]。

区块链的链状结构由多个区块连接而成,如图 1 所示。完整的区块由区块头和区块体组成。区块头中记录着版本号、父区块哈希值、Merkle 根等信息,而区块体中记录着事务请求和具体操作。区块之间通过父区块哈希值连接成一条完整链。

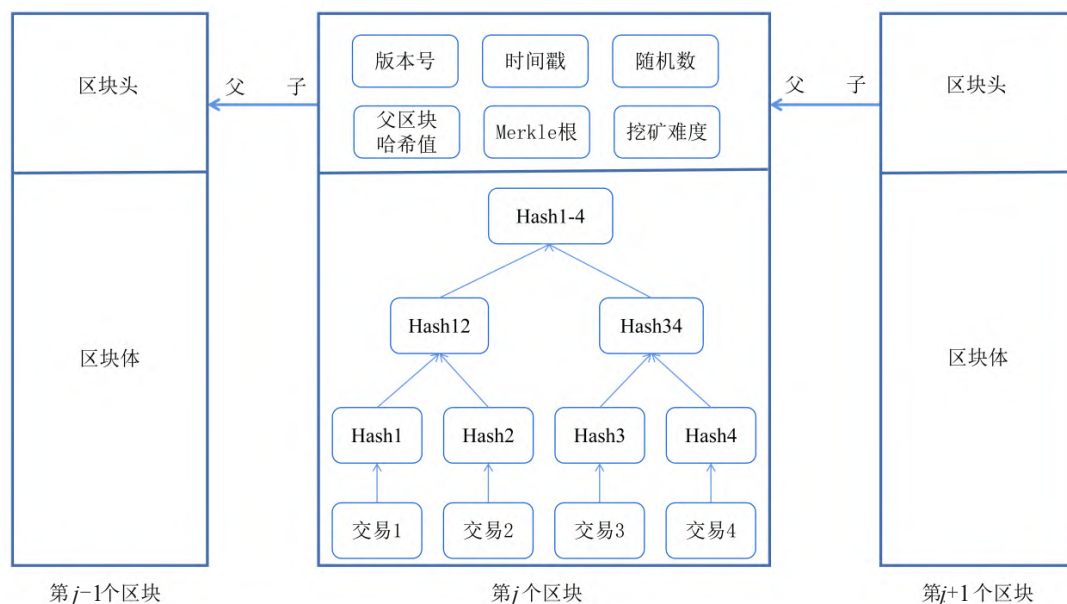


图 1 区块链的数据结构

Fig.1 Data structure of blockchain

1.2 PBFT 算法

在 1999 年, Miguel 等^[15]提出了 PBFT 共识算法,这是首个在实际应用中解决拜占庭将军问题的共识算法。该算法在拜占庭节点数量小于系统节点总数三分之一的前提下,能够确保最终结

果的正确性和一致性. PBFT 主要由 3 个核心协议构成:一致性协议、视图更改协议以及检查点协议.

一致性协议保证诚实节点按照相同的顺序正确处理事务请求,并将一致结果返回给客户端. 这为系统的可靠性和稳定性提供了有力保障. 视图更改协议能够在共识失败时恢复系统的正常运作,维护系统的活性. 检查点协议帮助节点安全地清理日志中的缓存,降低存储开销. 同时,它还帮助故障节点同步至最新状态,增强系统的容错能力.

一致性协议作为 PBFT 算法的核心组成部分,其包含预准备(pre-prepare)、准备(prepare)及确认(commit)3 个阶段. 这种 3 阶段共识协议的共识过程如图 2 所示.

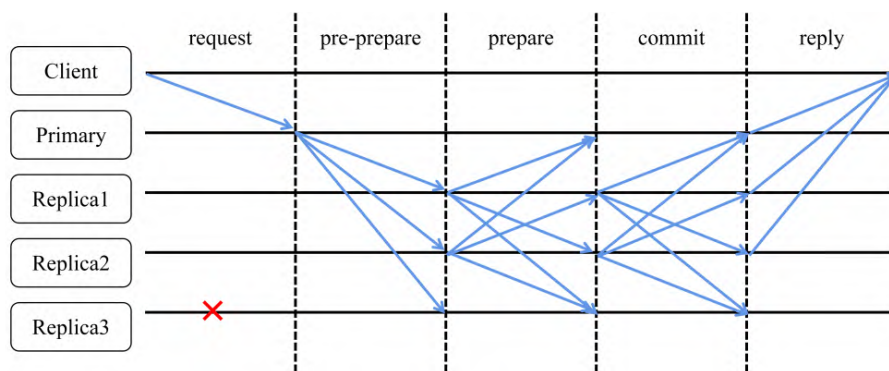


图2 PBFT 共识过程

Fig.2 Consensus process of PBFT

PBFT 的简要流程如下:

1) request 阶段是客户端发送请求消息 $\langle \text{REQUEST}, \text{me}, \text{ts}, \text{client} \rangle$ 给主节点. 其中: me 是客户端的事务请求; ts 是时间戳; client 是客户端号.

2) pre-prepare 阶段是主节点将未共识的事务请求状态改成为 pre-prepare, 并分配唯一的序列号, 广播消息 $\langle \text{PRE-PREPARE}, \text{view}, \text{key}, \text{D}(\text{me}) \rangle$. 其中: view 是视图号; key 是序列号; $\text{D}(\text{me})$ 是 me 的哈希值.

3) prepare 阶段是收到主节点的 PRE-PREPARE 消息, 备份节点验证通过后, 向广播消息 $\langle \text{PREPARE}, \text{view}, \text{key}, \text{D}(\text{me}), \text{id} \rangle$. 其中: id 是节点编号; f 是系统中拜占庭节点的数量. 当节点接收超过 $2f+1$ 条验证通过的 PREPARE 信息后进入下个阶段.

4) commit 阶段是节点互相广播消息 $\langle \text{COMMIT}, \text{view}, \text{key}, \text{S}(\text{me}), \text{id} \rangle$. 其中: $\text{S}(\text{me})$ 为节点签名集合. 当节点接收超过 $2f+1$ 条验证通过的 COMMIT 消息后进入 reply 阶段.

5) reply 阶段是节点后向客户端发送回复消息 $\langle \text{REPLY}, \text{view}, \text{ts}, \text{client}, \text{key}, R \rangle$. 其中: R 为事务请求的执行结果.

当客户端接收到 $f+1$ 个消息后表明该请求已经达成共识.

1.3 VRF

可验证随机函数 VRF 具备可验证性、随机性、唯一性等特点. 每个节点均具备独一无二的公私钥对 (公钥 pk_i 与私钥 sk_i). 在输入相同的 sk_i 和种子 seed 的情况下, VRF 计算函数

VRF_Evaluate会产生相同的 vrf_hash 和 vrf_proof . 其中: vrf_hash 是一个由 sk_i 和 seed 作为参数因子生成固定长度的伪随机字符串; vrf_proof 则是一份能够证明 vrf_hash 由 sk_i 和 seed 产生的证据. VRF 验证函数 VRF_Verify 能够在不对 sk_i 进行公开的情况下, 利用对应的 pk_i 进行验证. VRF_Verify 的输出结果为布尔值, 若验证通过, 则输出 true . 反之, 输出 false .

2 MVBFT 算法

本文所提出的 MVBFT 算法是对 PBFT 算法的改进. 主要包括以下 5 个主要方面:

1) 构建了一种流水线式结构, 引入节点信誉值, 使用新的节点角色划分方法, 以及优化了共识流程和视图更改协议. 流水线式结构通过定义共识阶, 对区块链的区块结构进行优化, 实现多轮共识并发执行.

2) 节点信誉值用于评估节点行为和状态, 解决如何计算和恢复节点信誉值的问题, 确保节点行为的可靠性.

3) 节点角色划分利用改进的 VRF 算法选择和验证节点角色, 确保了节点角色的随机性和可验证性, 提高系统的安全性和公平性.

4) MVBFT 算法针对 PBFT 算法的一致性协议进行优化, 调整网络拓扑结构, 有效减少了通信开销, 提高了共识过程的效率.

5) 共识阶完成或超时将触发视图更改协议, 进行共识阶视图切换, 维护系统的运行.

整体流程上, MVBFT 算法与 PBFT 算法基本一致, 未阐述部分保持原有设置.

2.1 流水线式结构

把一种将多轮共识有效地融合为一个连贯且更高效共识流程的机制定义为共识阶. 当共识阶启动时, 系统会一次性选出多个节点来担任每轮共识的主节点, 可以降低频繁选举主节点所带来的额外通信开销. 如图 3 所示, 在同一共识阶下, 新一轮共识的开始不必等待上一轮共识的完全结束, 而只需在交易收集阶段完成后即可启动, 使得多轮共识能够并行执行, 从而实现了流水线式共识. 这种设计不仅降低了主节点故障或恶意行为对整体共识效率的负面影响, 还显著减少了交易确认的延迟, 提升了系统的整体吞吐量和响应速度, 同时增强算法的安全性.

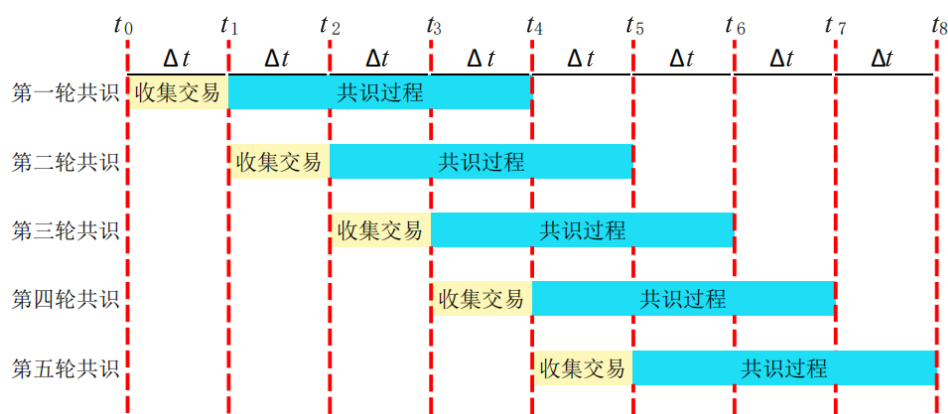


图3 MVBFT的流水线式结构

Fig.3 Pipelined structure of MVBF

为了实现流水线式共识,对区块链的区块结构和数据结构进行了优化,如图4和图5所示.

版本号	时间戳	本阶的编号 p	本阶的 $seed_p$
主节点的 pk_i	排名序号num	本轮的编号 v	本轮的 $seed_v$
前一共识阶哈希值 $hash_p$		Merkle根	
Merkle树			

图4 新的区块结构

Fig.4 New block structure

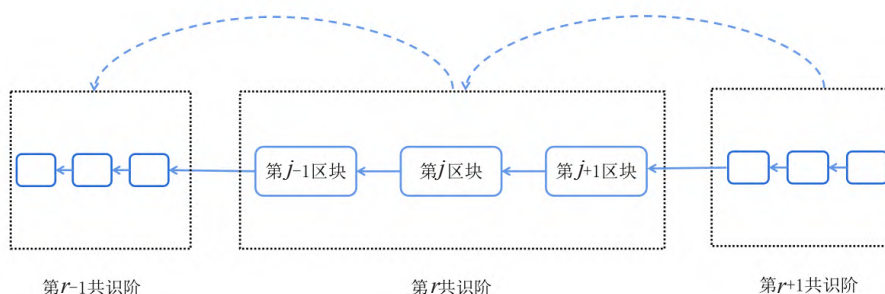


图5 新的数据结构

Fig.5 New data structure

在图4中,新的区块结构中添加了主节点公钥 pk_i 、排名序号 num 、共识阶编号 p 、种子 $seed_p$ 、本轮共识编号 v 和种子 $seed_v$. 同时,对哈希指针也做了修改,将原来存放的前一区块哈希值替换成前一共识阶哈希值 $hash_p$.

在图5中,同一共识阶下的区块根据 num 连成链作为一个整体,后一共识阶中第一个上链区块根据 $hash_p$ 连接到前一共识阶中最后一个上链的区块,从而连成一条完整的链,确保了区块链的可追溯性和不可篡改特性.

2.2 节点信誉值

2.2.1 节点信誉值计算

提出的节点信誉值 ($Credit_i$), 根据节点历史信誉值和共识过程中的行为表现进行计算. 节点的初始信誉值为 $Credit_{init}$, 在共识阶开始时会动态更新节点 i 的 $Credit_i$, 即

$$Credit_i = \alpha Credit'_i + \beta Pri + \gamma Con. \quad (1)$$

其中: $Credit'_i$ 为节点 i 的历史信誉值, 它反映了节点 i 在过去共识过程中的行为表现, 对新一轮信誉值的更新具有重要影响. Pri 代表对主节点的工作评价. 如果节点 i 是本轮共识的主节点并且成功达成共识, 那么 Pri 为 1; 如果节点 i 是本轮共识的主节点但未能达成共识, 那么 Pri 为 -1; 如果节点 i 不是本轮共识的主节点, 那么 Pri 为 0. 主节点在共识过程中担任领导者的角色, 需要引

导其他节点对交易达成一致共识.如果共识成功,主节点将获得奖励;如果共识失败,将受到惩罚.Con代表对共识节点的工作评价.如果达成共识,那么Con为1;否则,Con为-1.Con旨在鼓励共识节点诚实且积极地参与共识.最后,权重参数 α 、 β 和 γ 可根据系统的实际需求进行调整.

MVBFT引入了黑名单系统,将拜占庭节点进行标记并在全网范围内公布.一旦节点被列入黑名单,它们将无法参与共识过程和信誉值的更新.

2.2.2 信誉值恢复机制

为了预防节点因信誉值过高而导致寡头现象,设计了一种节点信誉值恢复机制.该机制旨在维护全网非黑名单系统节点的信誉值在离散范围 $[0, \text{Var}_{\max})$ 内.在完成计算全网节点的信誉值后,利用全网节点的信誉值计算全网非黑名单系统节点信誉值的均值Ex和方差Var,即

$$\text{Ex} = \frac{\sum_i^n \text{Credit}_i}{n}, \quad (2)$$

$$\text{Var} = \frac{\sum_i^n (\text{Credit}_i - \text{Ex})^2}{n}. \quad (3)$$

当Var大于或等于 $\text{Var}_{\max}$ 时,节点信誉值恢复机制将被触发.通过将信誉值超过Ex的节点进行信誉值缩小,以降低节点信誉值的离散程度.节点信誉值恢复机制更新节点 i 的 Credit_i ,表示为

$$\text{Credit}_i = \frac{\text{Credit}_i}{\text{Ex}}. \quad (4)$$

2.3 节点角色划分

在MVBFT算法中,节点角色被划分为主节点、共识节点和备份节点.为了提高系统的效率 and 安全性,算法引入了黑名单系统,将有作恶行为的节点标记为备份节点,只与共识网络保持同步而不参与共识过程,降低对系统的影响.主节点和共识节点的选取是通过节点信誉值和可验证随机函数(VRF)抽签来确定.

共识网络中设置了 N 个节点,并且每个共识阶包括 M 轮共识.在共识阶启动时,从非黑名单系统中随机选取 M 个节点加入到主节点集 Set_p ,并将未被选中的节点加入到共识节点集 Set_c . Set_p 中的节点按照排名序号num依次担任每轮共识的主节点,带领 Set_p 中的其他节点和 Set_c 中的全部节点一起完成共识过程.

MVBFT算法的节点角色划分和选取过程具有可验证性、随机性和公平性,确保了公平公正的共识网络环境.通过将拜占庭节点标记为备份节点,并排除在共识过程之外,MVBFT算法拥有更高的公式效率和安全性.

2.3.1 主节点和共识节点选取

MVBFT采用VRF抽签选举主节点,旨在提升选取过程的安全性和公平性.通过将节点的信誉值作为VRF抽签的一个参数,能够有效防止恶意节点通过大量注册新节点以提高当选主节点的概率,从而抵御女巫攻击,并确保选出的主节点具备较高可靠性.

VRF密码抽签流程如算法1所示.节点 i 将 sk_i 和 Seed_p 代入VRF_Evaluate函数中计算在区间 $[0, 2^{\text{hashlen}}]$ 内均匀分布的哈希值vrf_hash.取 $e = \frac{\text{vrf_hash}}{2^{\text{hashlen}}}$, e 均匀地分布在 $[0, 1]$ 之间.然后,通过计

算节点信誉值 Credit_i 和总信用值的比值 $\mu = \frac{\text{Credit}_i}{\sum_i^n \text{Credit}_i}$, 得到节点 i 在正态分布情况下的期望值为 μ 和标准差为 0 的概率密度函数. 对该概率密度函数进行积分得到分布函数, 计算区间 I 并求出 e 落在区间 I 内时的 k 值. 其中

$$I = \left[\int_{-\infty}^k \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2}} dx, \int_{-\infty}^{k+0.01} \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2}} dx \right]. \quad (5)$$

算法1 VRF 抽签算法

输入: $\text{sk}_i, \text{seed}_p, \text{Credit}_i, \sum_i^n \text{Credit}_i$

输出: $\text{vrf_hash}, \text{vrf_proof}, k$

1: $\text{VRF_Evaluate}(\text{sk}_i, \text{seed}_p) \rightarrow (\text{vrf_hash}, \text{vrf_proof});$

2: $e = \frac{\text{vrf_hash}}{2^{\text{hashlen}}};$

3: $\mu = \frac{\text{Credit}_i}{\sum_i^n \text{Credit}_i};$

4: $k \leftarrow 0;$

5: while $e < \int_{-\infty}^k \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2}} dx$

6: $k = k + 0.01;$

7: end while

8: return $\text{vrf_hash}, \text{vrf_proof}, k$

在抽签结束后, 各个节点会相互广播抽签的结果. 接下来, 根据抽签所得的 k 值, 将节点从高到低进行排序, 同时为每个节点分配一个排名序号 num . 若存在 k 值相同的节点, 则节点 ID 编号较小的排名更靠前. 排序完成后, 选取排名前 M 的节点纳入 Set_p , 其余节点则加入 Set_c .

2.3.2 检验抽签结果和种子计算的正确性

在角色选举过程中, VRF_Evaluate 函数需要一个动态更新的 seed_p 作为输入, 使得主节点的选举具有不可预测性. 在每轮共识过程中, 主节点会使用随机数生成函数 VRF_Evaluate_Seed 生成一个 seed_v , 并将其打包到交易块中. 在一个共识阶中最多会生成 M 个 seed_v , 但只有第一个成功上链的交易块中的 seed_v 才能成为下个共识阶的 seed_{p+1} . 主节点 i 生成 seed_v 如式 6 所示, 其中参数包括私钥 sk_i , 种子 seed_p , 共识阶编号 p , 视图编号 v . 如

$$\text{VRF_Evaluate_Seed}(\text{sk}_i, \text{seed}_p, p, v) \rightarrow (\text{seed}_v, \text{vrf_proof_seed}_v). \quad (6)$$

主节点在收集交易阶段不仅是收集交易池中的事务信息, 还需要将抽签选举时产生的 vrf_hash 、 vrf_proof 以及计算种子时产生的 seed_v 、 vrf_proof_seed_v 一起打包到交易块中. 当共识节点接收到该交易块后, 需要按照以下 2 个步骤进行验证:

- 1) 验证签名正确性, 即验证交易块的签名是否有效, 以确保该块是由主节点生成的;
- 2) 检验抽签结果和种子 seed_v 计算的正确性, 即使用主节点的公钥 pk_i 进行验证, 以确保抽签结果和种子 seed_v 计算是正确的. 这个验证过程可以使用算法 2 来实现.

算法2 验证算法

输入: $pk_i, seed_p, Credit_i, \sum_i^n Credit_i, vrf_hash, vrf_proof, k, seed_v, vrf_proof_seed_v$

输出: true/false

1: if $VRF_Verify(pk_i, seed_p, vrf_hash, vrf_proof) = \text{true}$ and
 $VRF_Verify_seed(pk_i, seed_p, seed_v, vrf_proof_seed_v) \rightarrow \text{true}$

2: $e = \frac{vrf_hash}{2^{hashlen}};$

3: $\mu = \frac{Credit_i}{\sum_i^n Credit_i};$

4: $k' \leftarrow 0;$

5: while $e < \int_{-\infty}^{k'} \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2}} dx$

6: $k' = k' + 0.01;$

7: end while

8: if $k' = k$

9: return true;

10: end if

11: else

12: return false;

2.4 共识流程

MVBFT算法对PBFT算法的一致性协议进行了优化,将第3个阶段共识过程中的多对多通信替换为一对多和多对一的通信,以降低通信开销和提高共识过程的效率.此外,通过引入多主节点的流水线式共识,将视图变更协议放置在共识阶的结束阶段.这些改进旨在优化系统性能,确保系统的一致性和可靠性.

MVBFT算法的一致性协议包括以下5个阶段:收集交易(collecte)、分发投票(dispanse)、投票(vote)、统计投票(count)以及回复(reply).详细的共识过程如图6所示.

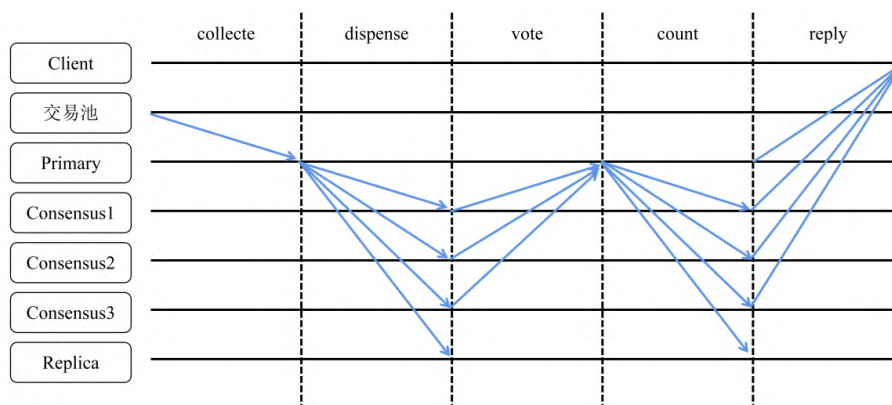


图6 MVBFT共识过程

Fig.6 Consensus process of MVBFT

以下是每个阶段的具体流程:

1)collecte 阶段是主节点从交易池中获取交易信息,将验证通过且无利益冲突的交易放入到交易块中.同时加入抽签信息和种子计算结果,供其他节点进行后续验证.

2)dispense 阶段是主节点将打包好的交易块向所有共识节点广播消息 $\langle \text{DISPENSE}, p, v, \text{seq}, \text{sq}, \text{sv}, d, t \rangle m$.其中:DISPENSE 为分发投票阶段的消息类型; p 为共识阶编号; v 为视图编号; seq 为交易块的标记编号; sq 为抽签数据; sv 为种子计算结果; m 为交易请求; d 为 m 的消息摘要; t 为时间戳.

3)vote 阶段是共识节点收到主节点广播的 DISPENSE 消息后,验证信息的正确性,包括对消息签名、抽签结果、种子计算结果的验证.如果所有验证都通过,节点将对交易块进行投票并给主节点发送投票消息 $\langle \text{VOTE}, p, v, \text{seq}, d, t, i \rangle$.其中:VOTE 为投票阶段的消息类型; i 为节点编号.否则将丢弃收到的交易块信息并进行举报.

4)count 阶段是主节点收集到超过三分之二的 VOTE 消息后,将投票结果和收集到的投票信息向全网广播消息 $\langle \text{COUNT}, p, v, \text{seq}, d, t, D \rangle \text{sm}$.其中:COUNT 为统计投票阶段的消息类型; sm 为主节点在投票阶段收到的投票信息; D 为 sm 的消息摘要.

5)reply 阶段是网络节点收到主节点的 COUNT 消息后,对信息进行验证.如果验证通过,则将该交易块加入到链上并向客户端发送投票结果消息 $\langle \text{REPLY}, p, v, \text{seq}, t, i, r \rangle$.其中:REPLY 为回复阶段的消息类型; r 为客户端交易请求的结果.否则将丢弃信息并进行举报.

当客户端收到超过三分之一的 REPLY 消息后,便可以确认交易的上链情况.

3 实验结果及分析

在 Intel i7-6700 处理器、20 GB 内存的硬件平台上进行实验,采用多线程模拟客户端和节点,通过 python 实现 MVBFT 算法和 PBFT 算法对比,其中的实验参数如表 1 所示.为了更好地验证 MVBFT 算法的有效性,我们对比了 PBFT 算法的通信开销、通信量和吞吐量.

表 1 实验参数

Tab.1 Experimental parameters

参数	设置
$\text{Credit}_{\text{init}}$	0
α	0.5
β	0.25
γ	0.25
Var_{max}	10
交易块中包含的交易数 Transactions	200
超时阈值 TO/S	10

3.1 通信量分析

3.1.1 通信量分析

设定共识网络中共有 N 个节点.在 PBFT 执行一致性协议过程中,pre-prepare 阶段的通信量

为 $N-1$, prepare 阶段的通信量为 $(N-1)^2$, commit 阶段的通信量为 $N(N-1)$. 在 PBFT 算法执行视图变更协议过程中的通信量为 N^2-1 . 设定触发视图变更协议的概率为 g , 则 PBFT 算法执行 M 轮共识需要的通信量为 $(2+g)MN^2-2MN-Mg$.

设定黑名单系统中的拜占庭节点数量为 F , 一个共识阶中包括 M 轮共识. 在下一个共识阶开始前, 节点之间通过多对多通信掌握全网的密码抽签结果, 该过程的通信量为 $N(N-1)$. 在 MVBFT 执行一致性协议过程中, dispense 阶段、vote 阶段和 count 阶段的通信量均为 $N-F-1$. 视图变更协议只在共识阶结束时执行, 所需通信量为 $3(N-1)$. 假设执行视图变更协议的概率为 h , 则 MVBFT 算法完成一个完整共识阶中的 M 轮共识需要的通信量为 $N^2+(3M+3h-1)N-3(MF-M-h)$.

3.1.2 通信量对比

令 PBFT 算法与 MVBFT 算法的通信开销比率为 R , 表示为

$$R = \frac{(2+g)MN^2-2MN-Mg}{N^2+(3M+3h-1)N-3(MF-M-h)}. \quad (6)$$

在实验中, 分别采用了不同的参数配置, 即 $g=1/2, h=1, M=10$; $g=0, h=1, M=10$; $g=0, h=0, M=10$; $g=1/2, h=1, M=5$; $g=0, h=1, M=5$; $g=0, h=0, M=5$, 结果如图 7 所示. 从图 7 可知, 随着节点数量的增加, PBFT 算法的通信开销相较于 MVBFT 呈现出明显的倍数增长. 此外, 共识轮次的增加也使得这一倍数进一步放大.

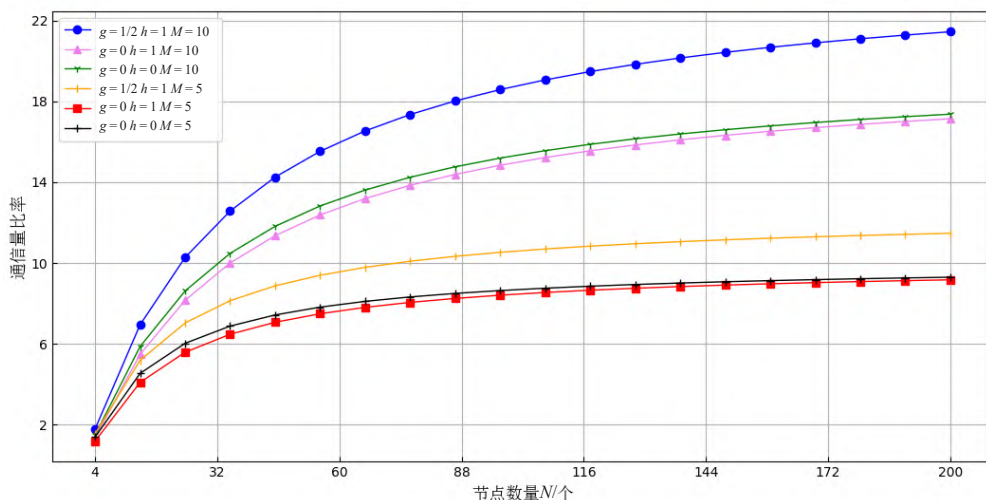


图7 MVBFT 共识过程

Fig.7 Consensus process of MVBFT

3.2 吞吐量分析

在共识领域, 吞吐量的大小能够直接反映算法的性能, 通常用每秒交易数 (transaction per second) 表示, 即每秒成功达成共识的交易数量. TPS 表达式为

$$TPS = \frac{\text{Transactions}_{\Delta t}}{\Delta t}. \quad (7)$$

其中: $\text{Transactions}_{\Delta t}$ 为出块时间内系统处理交易数; Δt 为出块时间. 吞吐量与交易块容量、节点

数量、网络延时等因素密切相关. 在固定交易块容量和网络延时的前提下,研究吞吐量与节点数量之间的关系,同时兼顾系统内部无拜占庭节点和有 1/3 拜占庭节点这两种情况. 结果如图 8,9 所示,对不同节点数量下 MVBFT 和 PBFT 的性能表现进行了分析.

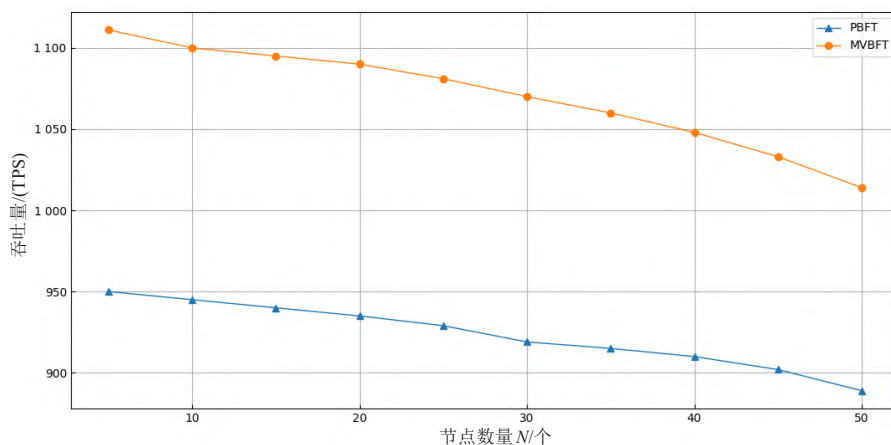


图8 无拜占庭节点时吞吐量比较

Fig.8 Comparison of throughput without Byzantine nodes

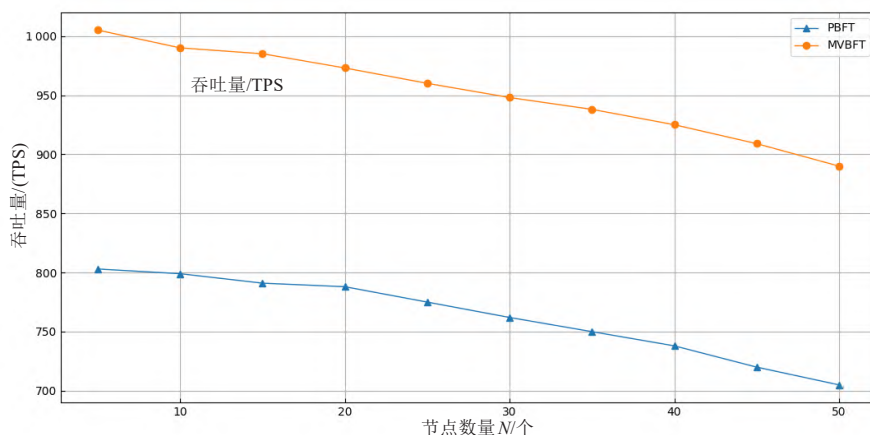


图9 有 1/3 拜占庭节点时吞吐量比较

Fig.9 Comparison of throughput with 1/3 Byzantine nodes

随着节点数量的增加,通信量和处理交易所需时间相应增长,导致共识效率逐步下滑,吞吐量减少.从图8与图9的对比中可观察到,无拜占庭节点的系统吞吐量较有 1/3 拜占庭节点的系统吞吐量更高.并且在两种情况中,MVBFT 的吞吐量均优于 PBFT.因此,MVBFT 性能表现优于 PBFT.

4 结论

基于 PBFT 算法,提出一种改进的拜占庭容错共识算法 MVBFT.定义共识阶的概念,将多轮共识划入同一个共识阶中,实现流水线式共识,提高共识效率.利用信誉模型和可验证随机函数 VRF 选取多个主节点,维护系统安全性.通过改变网络拓扑结构优化 PBFT 的一致性协议,减少通信量.实验结果表明,在相同的条件下,与 PBFT 算法相比,MVBFT 的通信开销更低,吞吐量更大.

参考文献:

- [1] 袁勇,王飞跃. 区块链技术发展现状与展望[J]. 自动化学报,2016,42(4):481-494.
- [2] 傅丽玉,陆歌皓,吴义明,等. 区块链技术的研究及其发展综述[J]. 计算机科学,2022,49(增刊1):447-461.
- [3] 李雅楠. 基于拜占庭容错的区块链共识算法研究[D]. 张家口:河北建筑工程学院,2022.
- [4] CASTRO M, LISKOV B. Practical byzantine fault tolerance and proactive recovery[J]. ACM Transactions on Computer Systems, 2002, 20(4): 398-461.
- [5] 袁勇,倪晓春,曾帅,等. 区块链共识算法的发展现状与展望[J]. 自动化学报,2018,44(11):2011-2022.
- [6] KOTLA R, ALVISI L, DAHLIN M, et al. Zyzzyva: Speculative byzantine fault tolerance[J]. ACM Transactions on Computer Systems, 2009, 27(4): 45-58.
- [7] LI Y X, QIAO L, LV Z H. An optimized byzantine fault tolerance algorithm for consortium blockchain[J]. Peer-to-Peer Networking and Applications, 2021, 14(5): 2826-2839.
- [8] 陈子豪,李强. 基于K-medoids的改进PBFT共识机制[J]. 计算机科学,2019,46(12):101-107.
- [9] 高娜,周创明,杨春晓,等. 基于网络自聚类的PBFT算法改进[J]. 计算机应用研究,2021,38(11):3236-3242.
- [10] 张雷. 基于实用拜占庭容错改进的多主节点共识机制研究[D]. 沈阳:辽宁大学,2022.
- [11] 刘鑫磊. 基于流水线的区块链共识算法研究与实现[D]. 郑州:河南工业大学,2023.
- [12] MICALI S. Algorand: The efficient and democratic ledger[EB/OL]. (2016-08-24) [2024-01-22]. <https://arxiv.org/abs/1607.01341>.
- [13] CHEN J, MICALI S. Algorand: A secure and efficient distributed ledger[J]. Theoretical Computer Science, 2019, 777: 155-183.
- [14] GILAD Y, HEMO R, MICALI S. Algorand: Scaling byzantine agreements for cryptocurrencies[C]//Proceedings of the 26th ACM Symposium on Operating Systems Principles. Shanghai: ACM, 2017: 51-68.
- [15] MICALI S, RABIN M, VADHAN S. Verifiable random functions[C]//Proceedings of the 40th Annual Symposium on Foundations of Computer Science. New York: IEEE, 1999: 120-130.
- [16] 任永旺,段红军,王振飞,等. 融合可验证随机函数和门限签名的拜占庭容错共识算法[J]. 小型微型计算机系统,2022,43(7):1484-1493.
- [17] 王博超. 一种结合DPoS与Vrf的分组Pbft共识算法[J]. 信息技术与信息化,2022,12:136-140.
- [18] 黄保华,彭丽,赵伟宏,等. 基于可验证随机函数的实用拜占庭共识算法[J]. 计算机科学,2023,50(S1): 737-742.
- [19] XU H, LONG Y, LIU Z Q, et al. Dynamic practical byzantine fault tolerance[C]//IEEE Conference on Communications and Network Security. Beijing: IEEE, 2018: 1-8.
- [20] 张仕将,柴晶,陈泽华,等. 基于Gossip协议的拜占庭共识算法[J]. 计算机科学,2018,45(2):20-24.
- [21] 曾诗钦,霍如,黄韬,等. 区块链技术研究综述:原理、进展与应用[J]. 通信学报,2020,41(1):134-151.
- [22] HAKAK S, KHAN W Z, GILKAR G A, et al. Recent advances in blockchain technology: A survey on applications and challenges[J]. International Journal of Ad Hoc and Ubiquitous Computing, 2021, 38(1): 82-100.

[责任编辑:姜生有]